

github 기본사용법

1 Git 개요



Git은 코드의 변경 이력을 추적하고 관리하는 분산 버전 관리 시스템으로 프로그래머들이 코드를 안전하게 저장하고 협업하는 데 필수적인 도구입니다.

주요 특징:

- 파일의 변경 사항을 시간순으로 기록
- 여러 사람이 동시에 작업 가능
- 이전 버전으로 쉽게 되돌리기 가능
- 브랜치를 통한 독립적인 작업 환경 제공

문제 상황	Git이 해결하는 방식
파일 여러 버전이 섞여서 혼란	commit으로 버전 단위 저장
실수로 코드 삭제	이전 버전으로 3초 만에 복구
여러 명이 동시에 수정	branch로 각각 작업 후 merge
백업 필요	GitHub에 push 해서 저장

Git을 사용하지 않으면, "사람이 파일을 기억하고 관리"해야 한다.
실수·혼란·충돌·배포사고가 반복되는 비효율적인 팀이 된다.
Git은 단순한 도구가 아니라 "팀이 살아남기 위한 최소한의 안전장치"다.

창시자 Linux Torvalds가 지은 이름으로, 영국 속어로 "멍청한 놈"이라는 뜻도 있고, 그냥 발음하기 쉬운 3글자 단어로 지었다고 합니다.

○ Git의 역사

2005년 탄생

- 리눅스 커널 개발자 Linus Torvalds가 만듦
- 당시 사용하던 BitKeeper라는 상용 버전 관리 시스템이 무료 사용을 중단하면서 필요에 의해 개발
- 단 **2주 만에** 첫 버전을 완성했다는 유명한 일화가 있음

개발 목표

- 속도가 빠를 것
- 간단한 설계
- 비선형 개발(수천 개의 병렬 브랜치) 지원
- 완전한 분산 시스템
- 리눅스 커널 같은 대규모 프로젝트 처리 가능

현재

- 전 세계에서 가장 널리 사용되는 버전 관리 시스템
- GitHub, GitLab 등의 플랫폼으로 생태계 확장
- 오픈소스 및 상업 프로젝트 모두에서 표준처럼 사용됨

○ Git & CI/CD

| CI/CD = 코드를 Git에 push하면, 서버까지 자동으로 배포되는 시스템.

✓ CI = Continuous Integration (지속적 통합)

- 코드를 Git에 올리면
- 자동으로 테스트 + 빌드 + 검증을 해주는 과정

✓ CD = Continuous Delivery / Continuous Deployment (지속적 배포)

- CI가 끝난 후
- 서버나 클라우드에 자동으로 배포해주는 과정

CI/CD가 산업 표준이 되면서, Git은 선택이 아닌 **필수**가 되었습니다. CI/CD를 하려면 Git을 알아야 하고, Git을 제대로 쓰려면 CI/CD 개념을 이해해야 합니다

CI/CD는 버전 관리가 필수

- 자동 배포하려면 코드 변경 이력을 추적해야 함
- 문제 발생 시 이전 버전으로 롤백 필요
- Git이 이를 가장 잘 지원

Git이 사실상 표준

- 대부분의 CI/CD 도구가 Git을 기본으로 설계됨
- GitHub Actions, GitLab CI/CD 등 주요 플랫폼이 Git 기반
- Git 없이 CI/CD 하기가 오히려 어려워짐

브랜치 전략과 자동화의 결합

- 브랜치별로 다른 환경에 자동 배포
- `develop` → 개발 서버, `main` → 프로덕션
- Git의 브랜치 기능이 CI/CD 파이프라인과 완벽하게 맞아떨어짐

협업의 필수 요소

- CI/CD로 여러 개발자가 동시 작업
- Git 없이는 코드 충돌 관리 불가능
- Pull Request + 자동 테스트 = 현대 협업의 표준

Git 생태계의 주요 플랫폼

핵심 차이점:

- **Git** = 버전 관리 도구 (프로그램)
- **GitHub/GitLab/Bitbucket** = Git 저장소를 온라인에서 호스팅하고 협업 기능을 제공하는 서비스 (플랫폼)

- **GitHub**

- 가장 유명하고 큰 Git 호스팅 플랫폼
- Microsoft 소유
- 오픈소스 프로젝트의 중심지
- 소셜 기능(팔로우, 스타 등)이 강함

전 세계 개발자들이 코드(project)를 올려서 저장하고, 버전관리(git)를 하고, 함께 개발(협업)하는 플랫폼으로 다음의 세가지가 합쳐져 있는 서비스임.

● 프로젝트 저장소(Repository) / ● 버전관리 시스템(Git) / ● 협업 플랫폼

- **GitLab**

- GitHub의 강력한 대안
- CI/CD(자동 배포) 기능이 내장되어 강력함
- 자체 서버에 설치 가능(Self-hosted)
- DevOps 전체 라이프사이클 지원

- **Bitbucket**

- Atlassian 소유 (Jira, Confluence 만든 회사)
- Jira와의 연동이 좋음
- 소규모 팀에게 무료 private 저장소 제공

- **Gitea / Forgejo**

- 가볍고 자체 호스팅하기 쉬운 오픈소스
- 개인/소규모 팀용

시나리오: "Git 없이 개발하던 팀의 하루"

등장인물

- **A개발자(준수):** 기능 개발 담당
- **B개발자(미나):** 버그 수정 담당

- 팀장님(현우): 배포 일정 관리

- 월요일 오전 — “파일버전 지옥 시작”

준수는 기능을 개발하며 아래처럼 파일을 계속 복사해서 버전을 쌓는다.

```
project_final_v1.py
project_final_v2.py
project_final_v3_realfinal.py
project_final_v3_realfinal_last.py
```

문제는...

어떤 파일이 최신인지 아무도 모름.

팀장: “최신 파일이 뭐야?”

준수: “음... 아마... realfinal_last... 인데요?”

- 월요일 오후 — 실수로 덮어쓰기 사고

미나가 버그를 고치다가 ‘realfinal_last’를 실수로 저장하며 일부 코드를 날린다.

문제: 이전 버전이 없음 → 되돌릴 방법이 없음

미나: “방금 수정한 게 전체 기능이 날아간 것 같은데...”

준수: “헉... 백업해둔 거 없어요?”

미나: “없죠. 그냥 파일 하나로만 개발했는데요...”

팀장: “이번 주 배포... 끝났다...”

- 화요일 — 두 개발자가 같은 파일 수정 → 충돌 지옥

준수가 **기능 A** 를 작업하고

미나가 **버그 B** 를 수정한다.

둘 다 **같은 파일**을 사용한다.

문제 발생

- 준수: 오전 버전
- 미나: 오전 파일 복사해서 버그 수정

- 오후에 서로의 수정이 사라짐

서로 덮어쓰며 "내 코드가 없어졌다", "네가 내 걸 덮었다" 싸움 발생.

팀장: "한 파일을 두 명이 동시에 수정하면 충돌 날 거라는 건 예측 가능했잖아..."

- 수요일 — 배포하려는데... 어느 파일인지 몰라 혼란

배포해야 하는데 최신 파일이 무엇인지 팀 전체가 모른다.

```
project_final_v3_realfinal_last2.py
project_final_v3_realfinal_last2_backup.py
project_final_v3_realfinal_last2_backup(2).py
```

- 준수는 "backup.py가 최신"이라고 함
- 미나는 "last2.py가 최신"이라고 함
- 실제로는 **backup(2).py**가 최신

팀장: "누가 좀 진짜 최신 파일 찾을 수 있어요?"

3명 모두 10분 넘게 탐색...

- 목요일 — 실험 코드가 섞여서 오류 발생

준수가 실험용으로 넣어둔 임시 코드:

```
# 테스트용 임시 코드
print("experiment...")
```

이게 배포 파일에도 그대로 들어가 실행됨.

팀장: "누가 테스트 코드 배포했어?"

준수: "아... 그거... 여러 버전이 섞여서 정리가..."

- 금요일 — 팀원 추가 배치 → 완전 카오스

신입 개발자 C가 들어오자마자 말한다:

C: "최신 버전 파일을 어디서 받아요?"

준수: "잠시만요. 제 USB에서... 아니, 미나님 컴퓨터에 최신이 있을 듯?"

결국 최신 파일을 찾는데 1시간 이상 소요.

신입: "이 팀은... Git 없나요...?"

팀장: "이번 주만 지나면 Git 도입하자..."

✓ Git이 없어서 생긴 핵심 문제 정리

문제	실제 피해
파일 버전 관리 불가	최신 파일 찾기 어려움, 시간 낭비
되돌리기 불가능	실수로 코드 삭제 → 복구 불가
협업 충돌	서로 덮어쓰기 → 기능/버그 수정 소실
실험 코드 섞임	배포 사고 발생
협업 onboarding 어려움	신입/외주/파트너가 파일 구조 파악 불가
백업 없음	컴퓨터 고장 = 프로젝트 날아감

- git 필수개념

개념	설명
Repository(Repo)	프로젝트 저장소 (이력 저장 공간)
Commit	변경 내용을 "스냅샷"처럼 기록하는 단위
Branch	독립된 작업 공간 (새 기능/실험용)
Merge	브랜치에서 작업한 내용을 본 흐름(main)에 합치기
Remote(GitHub 등)	내 Git 저장소를 인터넷에 올린 공간

Git은 크게 3개의 단계로 관리됨

- **Working Directory** → 실제로 파일을 수정하는 곳
- **Staging Area** → "이번 commit에 포함할 파일 목록"을 올려두는 준비 구역
- **Repository** → commit들이 저장되는 최종 저장소

가장많이 사용하는 git 명령어

명령어	의미
git init	git 시작
git status	지금 상태 보기
git add .	변경파일들을 staging area에 올리기
git commit -m "메시지"	버전 기록 만들기
git log	commit 기록 보기
git branch	브랜치 목록
git checkout -b newbranch	브랜치 만들고 이동
git merge 브랜치명	병합
git remote add origin URL	GitHub와 연결
git push	내 commit을 GitHub로 올리기
git pull	GitHub에서 최신버전 가져오기

2 GitHub Repository 생성

(1) GitHub 리포지토리(Repository) 개념

프로젝트 전체 폴더 = GitHub 리포

프로젝트를 저장하고 관리하는 공간(폴더 + 버전관리 + 협업 기능 포함)
 GitHub에서 하나의 프로젝트는 **리포(repo)** 로 만들어지고,
 그 안에 소스코드, 노트북, 문서, 폴더 등을 모두 저장한다.

○ 리포에 들어가는 것

- Python 코드 (`.py`)
- Jupyter Notebook (`.ipynb`)
- README.md (프로젝트 설명)
- 이미지, 데이터 파일
- 폴더 구조
- 설정 파일(.gitignore, requirements.txt 등)

○ 리포는 단순한 폴더가 아니다 → “버전 관리 기능” 포함

리포는 일반 폴더+파일 저장 장소가 아니라 각 변경사항을 모두 기록하고, 비교하고, 되돌릴 수 있는 구조임

└ 프로젝트의 모든 히스토리를 남기는 폴더

- 파일 수정: 기록됨
- 파일 이동: 기록됨
- 코드 삭제: 기록됨
- 새로운 파일 생성: 기록됨
- 커밋 메시지: 기록됨

○ 리포의 핵심 요소 5가지

1) Commit (커밋)

파일 변경 내용을 저장하는 기록 → 찍은 스냅샷이라고 보면 됨

2) Branch (브랜치)

한 프로젝트를 여러 갈래로 나누어 작업 → 실험용/메인용 등을 분리할 수 있음

3) Push (푸시)

로컬(내 PC, Codespaces)의 변경 → GitHub 서버로 업로드

4) Pull (풀)

GitHub 서버의 최신 내용 → 내 로컬/Codespace로 가져오기

5) Issues / Pull Requests

협업할 때 작업 내용, 요청, 리뷰 등을 관리

○ Public Repo vs Private Repo

종류	설명
Public	누구나 볼 수 있음 (오픈소스)
Private	본인과 초대한 사람만 볼 수 있음

(2)번은 pdf 04_깃허브시작하기.pdf 의 6페이지 ~10페이지 내용임

(2) GitHub 리포지토리(Repository) 생성

- www.github.com에 접속한 후 [Sign up]을 클릭
- 깃허브에서 사용할 이메일 계정을 입력한 후 [Continue]를 클릭
- 중복되는 메일 계정이 없으면 단계별로 비밀번호(password), 사용자 이름(username), 업데이트 소식을 받을지 여부를 묻는데 단계별로 필요한 정보를 입력하고 [Continue]를 클릭
사용자 이름은 영문자와 숫자만 사용할 수 있고, 단어가 2개이면 불임표(-)로 연결할 수도 있음
- 사용자 정보를 모두 입력하면, 'Verify your account'에 있는 문제를 풀고 [Create account]를 클릭
깃허브 계정을 만들 때 사용한 이메일 주소로 론치 코드(launch code)가 도착하는데, 이 코드를 확인해서 깃허브 화면에 입력해야 계정 만들기가 끝남
- 혼자 사용하는지, 여러 명이 함께 사용하는지 선택한 후 [Continue]를 클릭이후에 몇 가지 선택 사항이 나타나는데 그냥 [Continue]를 클릭해도 됨
- 깃허브에는 무료인 개인 계정과 유료인 팀 계정이 있는데, 깃허브를 공부하거나 개인 프로젝트라면 개인 계정을 사용하면 됨[Continue for free]를 클릭하는 것으로 계정 등록이 끝남
- 깃허브 화면을 처음 경험한다면 꽤 복잡해 보이지만, 아직 아무 저장소도 만들지 않았으므로 처음에는 깃허브 사용법과 깃허브의 최근 변경 사항 내용이 보임

지역 저장소와 원격 저장소

- 깃허브에서 버전을 관리할 때도 저장소를 만들어 사용해야 함
- 사용자 컴퓨터에 있는 저장소는 '지역 저장소(local repository)'라 하고, 깃허브에 있는 저장소는 '원격 저장소(remote repository)'라고 함
- 우선 지역 저장소를 만들어 작업한 후 그 내용을 원격 저장소로 올리고 변경 사항이 생길 때마다 원격 저장소에도 반영할 것
- 지역 저장소에서 원격 저장소로 커밋을 등록하는 것을 '푸시(push)'라고 함
지역 저장소를 거치지 않고 원격 저장소에서 커밋을 만들 수도 있는데, 원격 저장소의 변경 사항을 지역 저장소로 내려받는 것을 '풀(pull)'이라고 함
- 지역 저장소와 원격 저장소를 연결해 놓았기 때문에 지역 저장소의 변경 사항은 항상 원격 저장소로 올려 두어야 하고, 원격 저장소에서 무언가 변경 사항이 있다면 지역 저장소에 내려받아 두어야 함
- 이렇게 지역 저장소와 원격 저장소를 항상 같게 유지하는 것을 '동기화(synchronize)'라고 함

원격 저장소 만들기 - (1)

1. 깃허브에 로그인한 후 화면 오른쪽 위에 있는 [+]를 클릭하고 [New repository]를 선택
2. 저장소 이름을 비롯해서 필요한 항목을 기입하고 [Create repository]를 클릭

- ① **Repository name:** 저장소 이름을 입력
영문과 숫자, 언더바(_), 불임표(-) 등을 사용할 수 있음
공백이 포함되어 있으면 자동으로 불임표(-)로 바꿈
- ② **Description:** 저장소를 소개하는 간단한 설명을 입력
이 부분은 옵션이므로 반드시 입력하지 않아도 됨
- ③ **Public / Private:** 저장소를 공개로 할지 비공개로 할지 선택
공개 저장소는 주소만 알면 누구나 볼 수 있으나 만일 다른 사람에게 보여 주고 싶지 않은 프로젝트를 관리한다면 저장소를 만들 때 비공개(private)로 하면 됨

- ④ **Add a README:** 저장소를 소개하고 설명하는 내용을 작성하는 README 파일을 자동으로 만들려면 체크
여기에서는 체크하지 않도록 함
- ⑤ **Add .gitignore:** .gitignore 파일은 지역 저장소의 파일을 깃허브의 저장소로 푸시할 때 제외할 파일을 지정해 놓은 것 [▼]를 클릭한 후 어떤 언어와 관련된 것을 .gitignore 파일에 지정할지 선택
예를 들어 C++을 선택한다면 C++에서 사용하는 컴파일된 라이브러리나 실행 파일을 깃에서 무시하도록 .gitignore 파일을 자동으로 만들어 줌
- ⑥ **Choose a license:** 오픈 소스 프로젝트를 위한 저장소를 만들 때 해당 오픈 소스의 라이선스를 선택

3. 원격 저장소가 만들어지면서 즉시 저장소 페이지로 이동
아직 아무 파일도 들어 있지 않고, 맨 위에는 저장소에 접속하는 방법이 나타남
저장소에 접속할 때는 HTTPS 방식이나 SSH 방식 중에서 선택할 수 있는데 우선 우리는 HTTPS 방식을 사용해 접속할 것
4. 화면에 나타난 HTTPS 주소를 사용해 언제든지 깃허브 저장소에 접속할 수도 있고 파일을 올릴 수 있음
깃허브 원격 저장소 주소만 알고 있다면 어디에서든 지역 저장소를 백업하거나 다른 사람과 협업할 수 있음

원격 저장소의 HTTPS 주소는 다음과 같은 형태

```
https://github.com/아이디/저장소명
```

예를 들어 깃허브 계정이 jump2dev이고, 저장소 이름이 test-1이라면 주소는 다음과 같음

```
https://github.com/jump2dev/test-1
```

3 codeSpace를 통한 git Start

(1) codespace 개념

브라우저만 있으면 바로 실행되는 “온라인 VSCode 개발환경”으로 GitHub가 제공하는 클라우드(서버) 기반 개발 컴퓨터

- **바로 개발 가능:** 브라우저에서 VSCode가 바로 켜짐
 - python, node, docker 등 자동 설치됨
 - 즉시 코드 작성 가능
- **GitHub Repository와 완전히 연결됨**
 - 파일 만들기 → Git 변경사항 자동 반영
 - git add / commit / push 바로 가능
 - GitHub 서버에 바로 업로드됨
- **가상환경 자동 구성**
 - `.devcontainer` 설정만 만들면 Python 버전, 패키지, Docker 모두 자동 설치됨.
- **PC 사양과 무관**
 - 맥북/윈도우/저사양PC 상관없이 → 클라우드 컴퓨터에서 개발하는 것이라 빠름.
- **Jupyter Notebook 실행도 즉시 가능**

.ipynb 파일 실행하면 → Cloud에서 Jupyter Server 구동 → 브라우저에서 그냥 실행됨

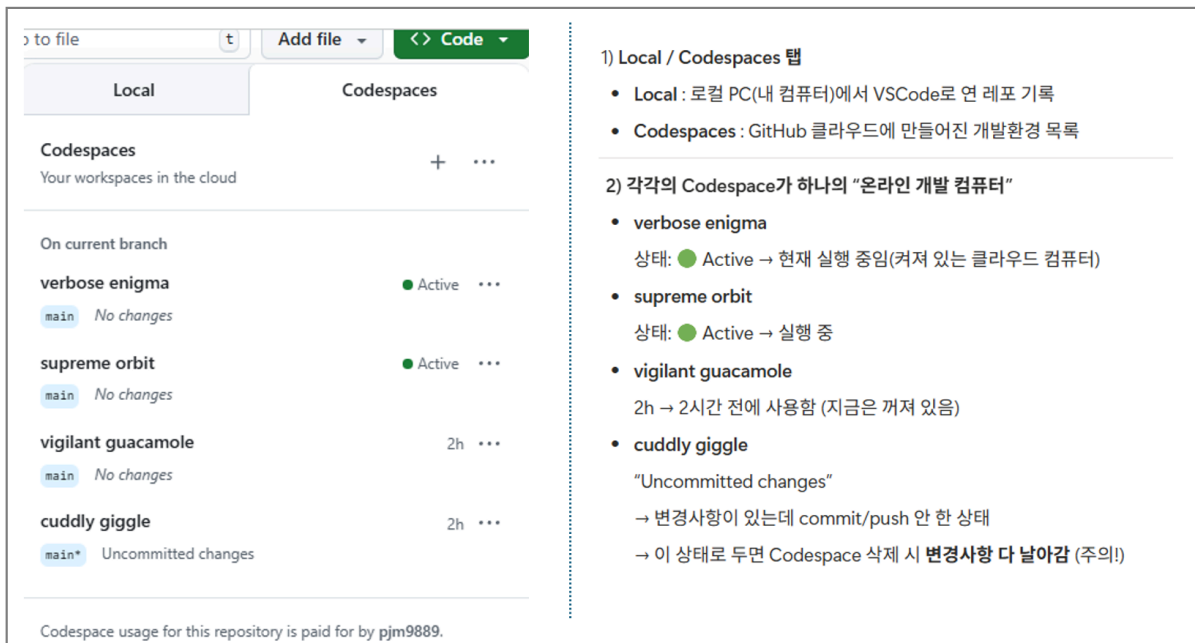
(2) codespace 실행 - readme.md 파일 수정

- 작성된 repo에서 <>Code 클릭 → Codespaces → + 눌러서 codespace 실행

- **예시화면:**

한 리포 안에 여러 개의 Codespace를 만들어서 사용하고 있고,

그중 2개는 현재 실행 중이며, 나머지는 최근에 사용했거나 변경사항이 남아 있는 상태



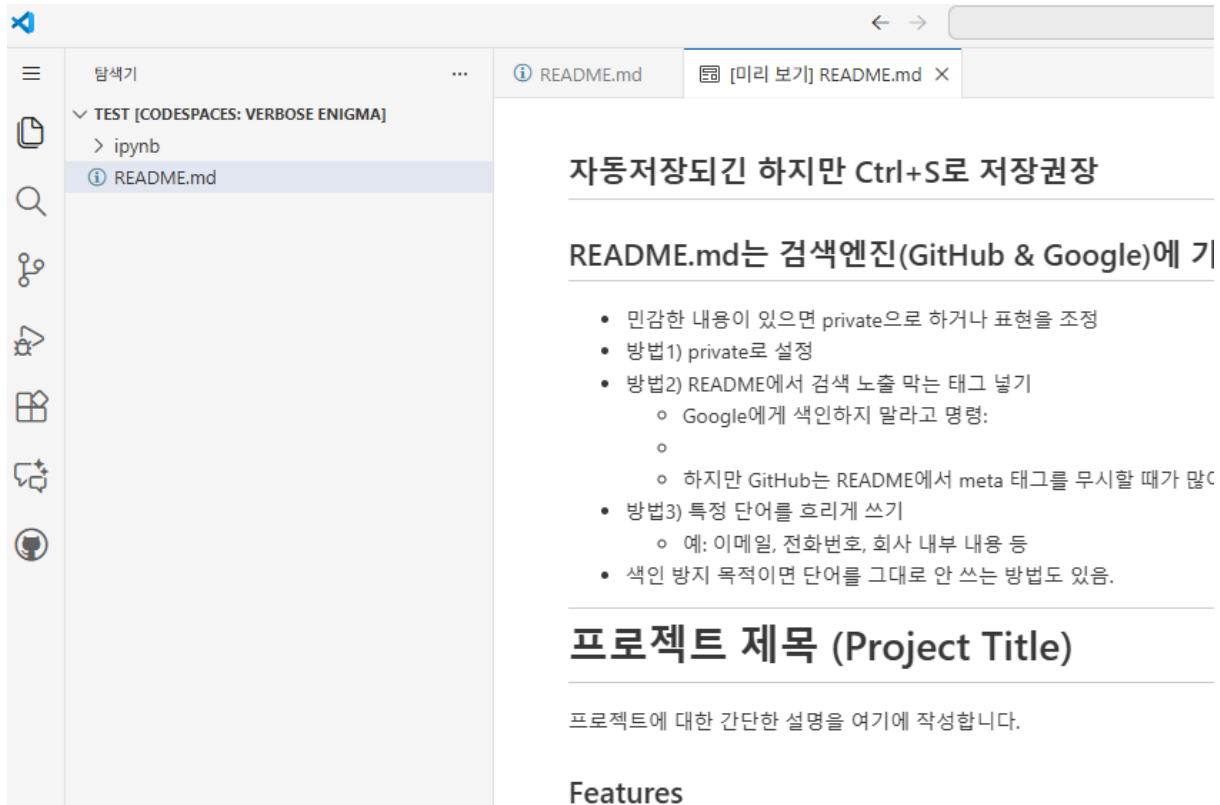
The screenshot shows the GitHub Codespaces interface. On the left, there's a sidebar with 'Local' and 'Codespaces' tabs. The 'Codespaces' tab is active, showing a list of workspaces. The main area displays a list of workspaces under the heading 'Codespaces' and 'Your workspaces in the cloud'. The list includes:

- verbose enigma**: Active (green dot), main branch, No changes.
- supreme orbit**: Active (green dot), main branch, No changes.
- vigilant guacamole**: Inactive (grey dot), 2h ago, No changes.
- cuddly giggle**: Inactive (grey dot), 2h ago, Uncommitted changes.

On the right side of the interface, there's a list of workspace names with their status and a brief description:

- 1) Local / Codespaces 탭**
 - Local**: 로컬 PC(내 컴퓨터)에서 VSCode로 연 레포 기록
 - Codespaces**: GitHub 클라우드에 만들어진 개발환경 목록
- 2) 각각의 Codespace가 하나의 "온라인 개발 컴퓨터"**
 - verbose enigma**: 상태: ● Active → 현재 실행 중임(켜져 있는 클라우드 컴퓨터)
 - supreme orbit**: 상태: ● Active → 실행 중
 - vigilant guacamole**: 2h → 2시간 전에 사용함 (지금은 꺼져 있음)
 - cuddly giggle**: "Uncommitted changes" → 변경사항이 있는데 commit/push 안 한 상태 → 이 상태로 두면 Codespace 삭제 시 **변경사항 다 날아감** (주의!)

○ readme.md 작성



○ 작성이 다 끝나면 하단의 터미널에서 다음작업진행해야 github에 기록 됨

- **Git의 3단계 구조 (중요)**

1. Working Directory → 실제 파일
2. Staging Area → git add로 올리는 곳
3. Repository → git commit + push로 저장되는 곳

git add . = Working → Staging

git commit = Staging → Repository

git push = Local Repository → GitHub server

Folder Structure

샘플 readme.md

https://github.com/KpmgFuture-Academy/fa02_fin_MODI

현재작업은 Codespace 컨테이너에만 저장된 상태

- GitHub에 저장(Push)해야 진짜로 보관됨
- Codespace가 끝나고도 기록을 남기려면 Git commit + push 필요함.
- 하단의 터미널에서
 - `git add .`
 - `git commit -m "작업 내용 저장"`
 - `git push`

- 주의사항

착각	실제 의미
<code>git add .</code> 하면 GitHub로 올라간다?	❌ Push해야 올라감
add 하면 저장된다?	❌ commit 해야 저장됨
add 하면 코드가 실행된다?	❌ 단지 "기록 준비"만 됨

`git add .` : 현재 폴더의 모든 변경사항을 "커밋할 준비 목록"에 올리는 명령

- 모든 변경 파일 올리기 : `git add .`
- 특정 파일만 올리기: `git add app.ipynb`
- 특정 폴더만 올리기: `git add ipynb/`

`git commit -m` 에서 `-m` : 커밋 메시지를 바로 명령어 옆에서 적겠다는 표시
 -m을 안하면 터미널이 자동으로 편집기(vim등)을 켜서 커밋메세지를 받음
 여러 줄 메시지를 넣고 싶으면: `git commit -m "ipybnb 자료 정리" -m "폴더 생성 및 이동"`

Codespaces에서는 commit이 매우 중요함

Codespaces는 "클라우드 임시 컴퓨터"라서
 → 삭제되면 파일도 함께 사라짐 → commit 안 했으면 데이터 복구 불가
 Codespaces 쓸 때는 commit이 거의 필수

commit 꼭해야 하는경우

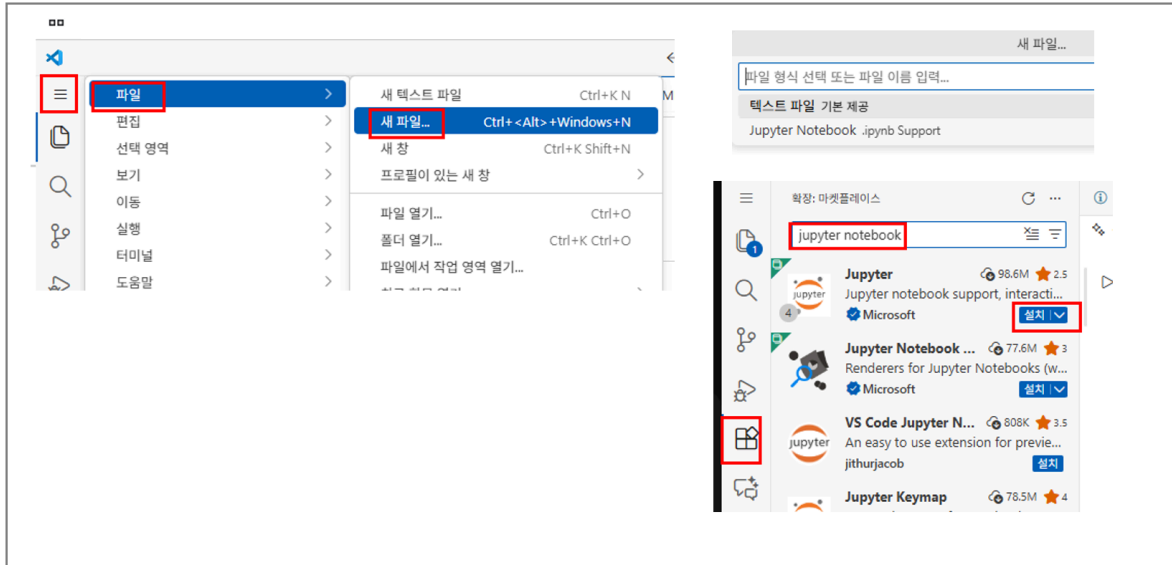
상황	commit 필요?	이유
GitHub에 올리고 싶을 때	✓ 반드시	push는 commit이 있어야 가능
작업 기록을 남기고 싶을 때	✓ 반드시	commit 메시지는 변경 히스토리
Codespace 지우기 전에	✓ 반드시	commit 안 하면 변경사항 다 날아감
팀 프로젝트	✓ 필수	기록/리뷰 위해 commit 필요

commit 안해도 되는 경우

상황	commit 필요 없음	이유
그냥 로컬에서 잠깐 테스트만 할 때	✗	Git 기록 필요 없음
파일을 만들었는데 Git으로 관리 안 하려고 할 때	✗	Git이 무시
코드를 버릴 예정이고 기록 필요 없을 때	✗	저장 의미 없음

(3) ipynb 파일 작성

주피터노트북을 실행하고 내용 작성뒤 → 파일-다른이름으로 저장(ctrl+shift+s)
 aapp.ipynb로 저장함



git add .

git commit -m "테스트"

```
@pjm9889 → /workspaces/test (main) $ git commit -m '테스트'
[main 06c1cdf] 테스트
1 file changed, 21 insertions(+)
create mode 100644 app.ipynb
```

- 21 insertions(+) : 새로 추가된 라인이 21줄 : Jupyter Notebook 파일은
 - 내부가 JSON 구조 / 보기엔 몇 줄 안 돼도 / 실제 저장 시 수많은 메타데이터(line) 들이 포함
 - 처음 저장하면 보통 수십~수백 줄이 insertions 로 잡힘
- Git은 파일의 변경을 "라인(line) 단위"로 계산함
 - 새 파일 생성 = 모든 라인이 insertions
 - 기존 파일 수정 = 변경된 줄 + 추가된 줄
 - 삭제된 내용은 deletion(-)으로 표시됨

(4) 폴더 생성 및 파일 이동

다음 작업을 터미널에서 실행합니다.

```
mkdir ipynb  
ls  
mv app.ipynb ipynb/  
ls  
git add .  
git commit -m "app.ipynb 파일 옮김, -> ipynb폴더로"  
git push
```